One advantage of the waterfall visualization is that we can tell, simply by visually inspecting all the spans in the trace, where most of the time is spent (in our case, in figure 8.3, the checkout service spends around half of the time in the billing phase). With this concrete motivational example of a microservices-based e-commerce app established, let's move on to a more formal definition of terms you will come across in the distributed tracing space.

8.1.2 Terminology

While there are differences in terminology across vendors, the following are terms that are usually well defined and understood across offerings. I'm following mostly the OpenTelemetry nomenclature (http://mng.bz/d1QN), especially where there is no consensus among distributed tracing solutions:

- A *span* represents a unit of work and, with it, captures a part of the execution of the request (compare with figure 8.2). This can happen within the scope of a service, such as the execution of a function in a service or across services. A special kind of span in a trace is the *root span*, which is created when the request enters the app. A span has an ID, and typically, it stores the ID of the trace it belongs to and the root span it was triggered from as well as payload-including metadata (think timestamps, tags, etc.). Note that in AWS X-Ray, this unit is called a segment.
- A *trace* is the end-to-end processing of a request in a distributed system, consisting of a group of spans that are temporally ordered. It has an ID and metadata.
- In the context of cross-service calls, the *trace context* captures all unique identifiers for individual traces and requests. This enables a trace to exist beyond a single service via a process called *context propagation*, where this trace context is forwarded in a network call. The calling service (parent) attaches all necessary data for the trace context to the request to the downstream (child) service, such as was the case in our example e-commerce app (figure 8.2) between the billing service and the third-party payment provider. Nowadays, with OpenTelemetry establishing itself as the standard, this propagation is usually done based on the Trace Context (https://www.w3.org/TR/trace-context/) W3C Recommendation (the standard way to use HTTP headers to propagate the context).
- To reduce the volume of data (number of traces and spans), you can make use of a technique called *sampling*. This allows you to selectively drop traces based on a characteristic including a fixed percentage or based on the nature of a trace—for example, if an error is present in one of the services, and with it we see that error fagged in the span representing the service.

Now that we have a basic understanding of distributed tracing terms and what they mean, let's look at some use cases.